



#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

Intel-Accel-for-LLM | Open Source Acceleration Library

KVShrink: KV Cache 多级缓存方案

传得快，存得多：Xeon DSA / QAT / AMX 加速 KV 传输、压缩与复用

github.com/intel/intel-accel-for-llm Bin Yang | 2026



KV Cache 走出 HBM：多级缓存已成业界标准

01 容量必须向外扩展

长上下文、多轮对话、多任务并发的 Agent 场景,使 KV Cache 规模远超 HBM 容量

GPU HBM | Active KV

实时推理 · 10–100 GB

02 按热度分层承载

活跃 KV 留在 HBM, 会话与历史下沉 DDR 与 Storage

HOST DDR | Host KV Cache

多轮会话 · Agent · 0.5–2 TB

03 已是主流推理框架标配

vLLM KV Connector、SGLang HiCache、LMCache、Mooncake 等

STORAGE | Persistent KV Cache

历史会话 · Agent · TB–PB

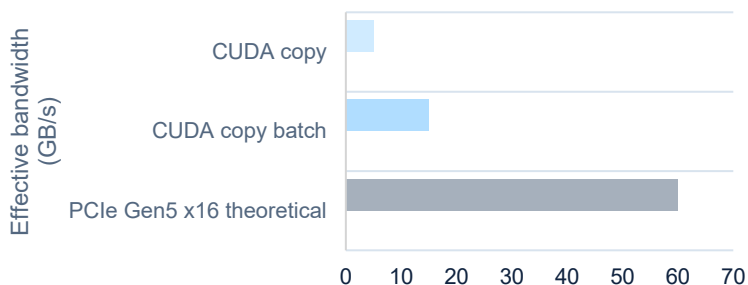
HBM 承载实时推理, DDR 与 Storage 承接会话与历史上下文

KV Cache 的双重挑战：传得快，也要存得多

01 碎片化传输

离散 KV pages 形成大量小块
常规 CUDA copy 难以吃满 PCIe 带宽

有效带宽有明显缺口 (32KB)

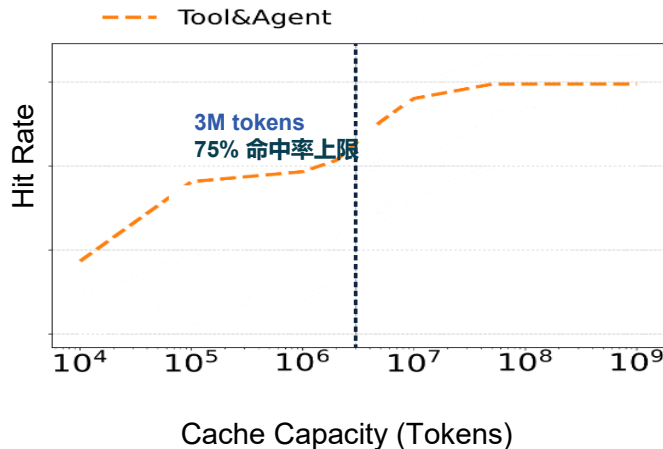


| RTX 6000D | PCIe Gen5 x16 |

Batching 有改善，但常规 CUDA copy 仍未充分利用 PCIe。

02 海量缓存

更高命中率，需要更大的历史 KV 缓存



Source: FAST'25 Mooncake slides ·
usenix.org/system/files/fast25_slides-qin.pdf

每 1M Tokens:

Kimi K3
~ 30 GB

MiniMax-M3
~ 150 GB

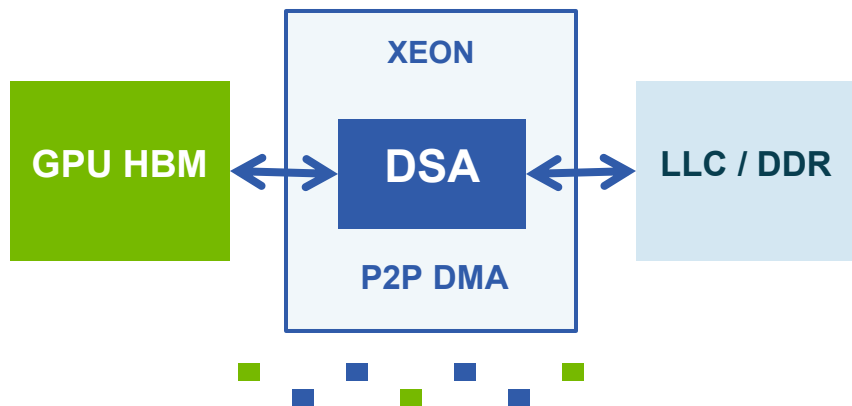
GLM5.2
~ 100 GB

Qwen3.6-27B
~ 70 GB

Source: docs.lmcache.ai
KV Cache Calculator

容量与带宽必须同时优化，才能把 KV 复用转化为更高的 GPU Productivity。

Xeon DSA 优化碎片化 KV 传输



Batch descriptors keep many fragments in-flight

DSA 独立搬运 KV，不占用 GPU SM

01 高并发 · 提升 PCIe 利用率

批量发起小块 DMA，用 in-flight 请求隐藏 PCIe 延迟。

02 DDIO · 数据直达 LLC

DMA 数据直接进入 Xeon LLC，便于后续处理就近访问，减少不必要的 DDR 带宽占用。

03 SVM · 无需预先 pin memory

使用用户态虚拟地址，省去预注册与锁页。

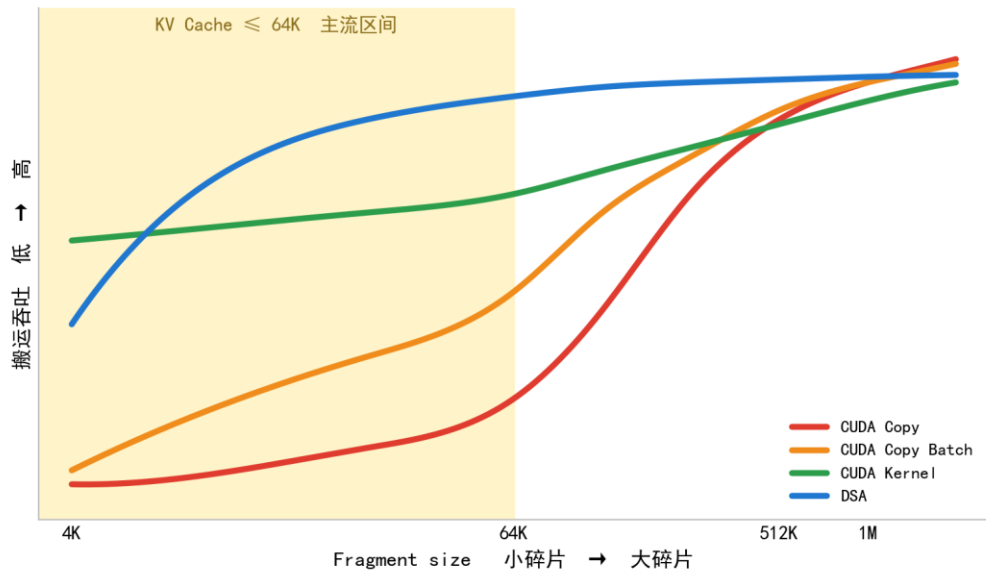
04 用户态提交 · 避免内核往返

ENQCMD / MOVDIR64B 启动 DMA，减少内核往返。

Xeon DSA 实现类 GPU-Direct DMA：不占用 GPU SM，批量 in-flight 搬运提升 PCIe 有效带宽。

DSA 搬运更快，且不占 GPU 算力

KV Cache 加载（H2D）：推理性能的关键路径



搬运效率成倍提升

对比 CUDA API 拷贝， ≤ 64 KiB 小包场景

搬运全程不占 GPU 算力

CUDA kernel 拷贝需占用 SM，而 prefill 阶段本就算力吃紧；DSA 由 Xeon 独立完成搬运，SM 全部留给模型计算。

主流负载正处 DSA 优势区间

推理引擎默认配置下，主流模型的分页 KV 多在 64 KiB 以下，如 Kimi-K3、DeepSeek-V4、GLM-5.2、Qwen3.8-2.4T/27B 等。

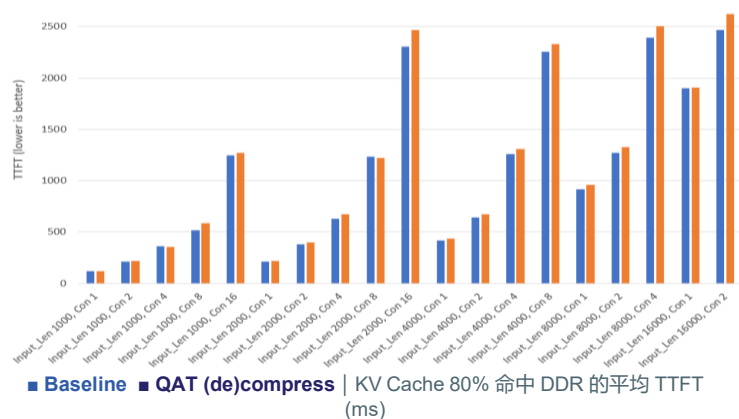
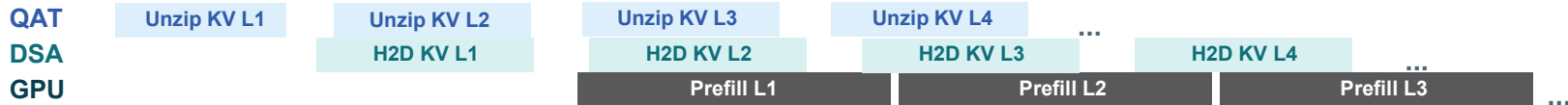
DSA 高并发搬运 KV Cache，提升 PCIe 利用率，同时释放 GPU SM 专注模型计算。

无损压缩：QAT 硬件卸载 KV Cache 压缩 / 解压

20%–30% 无损压缩率 $\approx 1.00\times$ TTFT，推理性能无损

压缩无精度损失，同等容量保存更多可复用 KV

逐层加载流水线



01 硬件卸载

压缩 / 解压跑在 QAT 专用硬件，吞吐远高于 CPU

02 逐层流水

与 GPU 逐层计算重叠，数据搬运与压缩 / 解压延迟均被隐藏

03 容量与命中率

同等 DDR / 存储缓存更多 KV Cache，命中率更高

04 I/O 更省

SSD 与分布式存储 I/O 更少，取数更快

用同样的存储承载更多可复用 KV Cache，把容量红利转化为更高的缓存命中率。

有损压缩：AMX 降熵 + QAT 压缩

先降低 KV 张量的信息熵，再由 QAT 无损压缩落盘，有损与否由策略门逐任务判定



~70% KV Cache 体积下降 **3×** 可缓存的历史上下文
同等 DDR / Storage 容量下，可复用上下文显著增加

01 AMX 降熵预处理

AMX 算子对 KV 做有损降熵，降低张量信息熵，使后续 QAT 无损压缩率大幅提升

02 KNN 策略门：按任务选路径

检索已验证任务中最相近的 K 个，对其压缩策略加权投票；置信度高则启用有损 profile，否则回退无损

03 硬件卸载

AMX / QAT 承担降熵与压缩，与 GPU 计算重叠，几乎不影响 TTFT

用同样的 DDR / Storage 承载更长的历史上下文，提升复用命中率。

KVShrink: Intel KV Cache 加速参考方案

vLLM × KVShrink



加速能力

DSA / QAT / AMX 加速 KV Cache 传输、压缩与复用

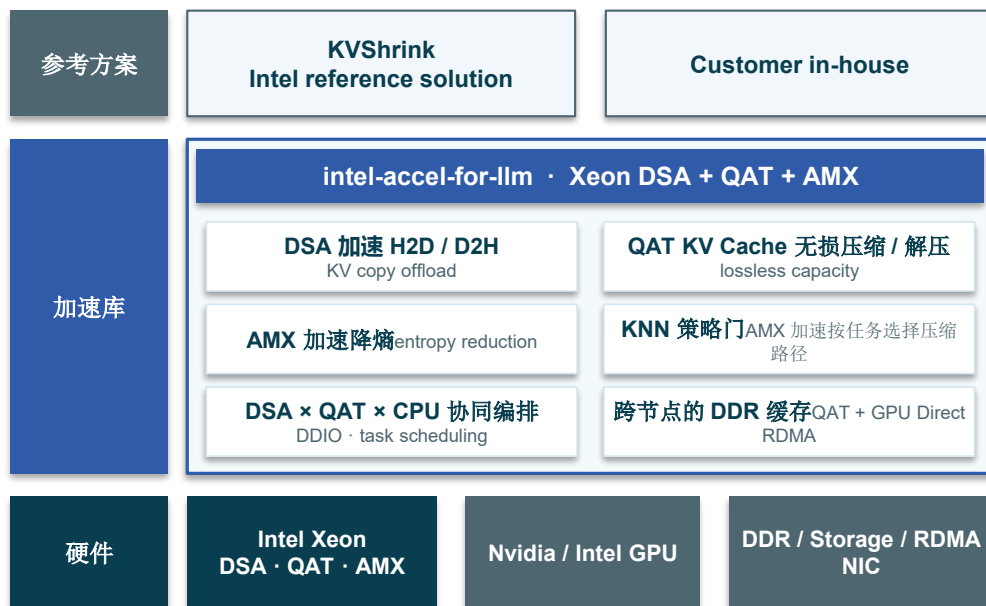
接入方式

vLLM 通过 KV Connector 接入

开源项目

<https://github.com/intel/intel-accel-for-llm>

架构



通过 KV Connector 集成 vLLM，由 intel-accel-for-llm 加速 KV Cache 数据面



China 2026

